

Milestone 1 — Task 1: Study Agentic AI Workflows

Project: AI Academic Project Mentor **Milestone:** 1 (Week 1-2) **Task scope:** Foundational research — no code, no build. Output is understanding that directly informs Task 2 (architecture/ER design).

1. Objective

Before designing the system architecture or database schema, you need a working mental model of how agentic AI systems are structured. This task has four required areas of understanding, as specified by the mentor.

2. Core Concepts

2.1 What is an AI agent?

An AI agent is **not** a single LLM call that returns an answer. It's an LLM wrapped in a loop that can:

1. **Observe** — take in some state (user input, prior results, tool output)
2. **Reason** — decide what to do next
3. **Act** — call a tool, make a decision, or produce output
4. **Repeat** — re-observe the result and continue until the task is complete

The defining trait is **autonomy across multiple steps** without a human re-prompting at each step.

2.2 What is a multi-agent system?

Instead of one large agent (or one giant prompt) trying to handle every responsibility, work is split across several narrower, specialized agents — each good at one job.

Why this matters for this project: A single "mega-prompt" trying to assess feasibility, define scope, recommend a tech stack, build a timeline, and flag risks all at once produces shallow, hard-to-debug output. Splitting into five focused agents (Feasibility, Scope, Tech Stack, Planning, Risk) keeps each agent's task narrow and its output verifiable.

Common topologies:

Pattern	Description	Fit for this project
Orchestrator-worker	One planner/router delegates to specialized sub-agents	Yes — matches the pipeline structure
Peer-to-peer	Agents negotiate directly with each other	No — unnecessary complexity here
Hierarchical	Nested orchestrators managing sub-orchestrators	No — overkill for this scope

2.3 How do agents communicate?

Two dominant patterns:

- **Message passing** — Agent A's output text becomes Agent B's input text directly, like a relay.
- **Shared state / blackboard** — All agents read from and write to one common object that accumulates fields as it moves through the pipeline.

Decision for this project: Shared state fits better. The five agents aren't holding a conversation — they're each enriching one growing project record (a `ProjectContext` object).

2.4 How are prompts/data passed between agents?

Handoffs should be **structured data (JSON)**, not free-text paragraphs. Agent A returns a structured object; that object is injected into Agent B's prompt as context; Agent B adds its own fields and passes the enriched object forward.

Example of the shared object growing through the pipeline:

```
json
```

```

{
  "raw_idea": "AI tool that recommends...",
  "feasibility": {
    "score": 7,
    "concerns": ["scope creep"]
  },
  "scope": {
    "in_scope": ["..."],
    "out_of_scope": ["..."]
  },
  "tech_stack": {
    "backend": "FastAPI",
    "reasoning": "..."
  },
  "plan": {
    "milestones": ["..."]
  },
  "risks": {
    "blockers": ["..."],
    "mitigations": ["..."]
  }
}

```

By the time the object reaches the Risk agent, it already contains the feasibility score, defined scope, and recommended stack — so risk assessment is scoped intelligently instead of guessing blind (e.g., flagging "no prior experience with recommended stack" as a risk).

3. Application to This Project's Pipeline

```

Student idea (raw, 2-3 lines)
    ↓
Feasibility Agent    → scores viability
    ↓
Scope Agent         → defines project boundaries
    ↓
Tech Stack Agent    → recommends stack + reasoning
    ↓
Planning Agent      → builds week-wise milestone plan
    ↓
Risk Agent          → flags blockers + mitigations
    ↓
Project Blueprint (final structured output)

```

Each stage reads the shared `ProjectContext` object built so far, adds its own fields, and passes it forward. This is the orchestrator-worker + shared-state pattern applied directly to your five required agents.

4. Why This Shapes Task 2 (Architecture Design)

- The shared `ProjectContext` object's shape is effectively your **future schema** for `Milestone`, `Risk`, and related tables — even though those tables aren't built until later milestones.
 - The ER diagram in Task 2 should leave room for this growing object without needing a redesign later.
 - The architecture diagram should show a placeholder "Agent Orchestration Layer" between the API and the database, even though it's empty in Milestone 1.
-

5. Study Actions (Definition of Done for Task 1)

- ☐ Read a short (15–20 min) explanation of how one real framework implements agent state handoff — e.g., LangGraph's "state" concept or CrewAI's "task output → next task input" pattern. Not for adoption — just to confirm the pattern in a real tool.
- ☐ Write, in your own words, why shared-state communication fits this project better than pure message-passing.
- ☐ Sketch (even roughly, on paper) the five-agent pipeline shown above before starting Task 2's formal diagrams.

Deliverable for Task 1: Personal understanding/notes only — no code, no diagrams yet (those are Task 2).